CS24301 — THEORY (3.0 CR) | MODULE IV NOTES | BIT MESRA | NEP SCHEME

Module 4: Intermediate Code & Runtime

Three-Address Code (Quadruples/Triples), Arrays, Backpatching & Activation Records

Module IV: Intermediate Code & Runtime Environment — Complete 9-Topic Syllabus Tracker

Topic 1: Complete Evaluation of Boolean Expressions

Topic 2: Partial Evaluation (Short-Circuit) of Boolean Expressions

Topic 3: Translation of Control Flow Constructs (`if-else`, `while`)

Topic 4: Resolution of Forward Jumps & Backpatching Algorithms

Topic 5: Resolution of Backward Jumps in Loop Constructs

Topic 6: Translation of Function Calls (`param`, `call`, `return_val`)

Topic 7: Translation of Function Returns & Caller Restoration

Topic 8: Memory Layout of Code and Data (Text, Static, Heap, Stack)

Topic 9: Activation Records (Stack Frame Anatomy: P-R-C-A-S-L-T)

Topic 1 & 2: Boolean Expressions (Complete vs. Short-Circuit Evaluation)

Evaluation Strategy	Operational Semantics	Key Advantages & Tradeoffs
1. Complete Evaluation	Evaluates all sub-expressions unconditionally before applying logical operators (e.g., bitwise `&` and `|`).	Evaluates side-effects in all operands; can cause runtime crashes on null-pointer guards (e.g., `p != NULL && p->val == 5`).
2. Partial Evaluation (Short-Circuit)	Stops evaluating as soon as the final truth value is guaranteed: • In `A && B`: If A is false, B is never evaluated. • In `A || B`: If A is true, B is never evaluated.	Highly optimized execution speed; safely handles null-pointer guards and range checks.

Topic 3 – 5: Control Flow Translation & Jump Resolutions

🏛️ Three-Address Code for `if (a < b) x = 1; else x = 2;`

```
    if a < b goto L1
    goto L2
L1: x = 1
    goto L3
L2: x = 2
L3: /* End of If-Else */
```

Backpatching Mechanics (`makelist`, `merge`, `backpatch`):

Forward Jumps: Jump targets point to instructions not yet generated. The compiler emits placeholder jumps (`goto ?`) and maintains lists of pending jump instructions.

Backward Jumps: Jump targets point to previously emitted labeled instructions (common in loop repeat steps). Target address is immediately known.

`makelist(i)`: Creates a new list containing index i.
`merge(p1, p2)`: Combines two jump lists p_1 and p_2.
`backpatch(p, target_label)`: Fills $target\_label$ as the destination for all jump instructions indexed in list p.

Topic 6 & 7: Translation of Function Calls & Returns

Standard Calling Sequence Three-Address Instructions: For high-level call `x = f(a, b);`:

```
param a
param b
call f, 2
t1 = return_value
x = t1
```

For function return `return y;`:

```
return y
```

Topic 8 & 9: Memory Layout & Activation Records

Figure 4.1: Runtime Memory Address Space Layout

🧠 **Memory Hook: Activation Record Anatomy (P-R-C-A-S-L-T)**

Field	Functional Role in Stack Frame
P — Parameters	Actual argument values passed by the caller.
R — Return Value	Space allocated to return computation result to caller.
C — Control Link	Dynamic link pointing to caller's activation record (restores stack frame on return).
A — Access Link	Static link pointing to enclosing lexical scope for resolving non-local variables.
S — Saved Status	Saved machine state (Program Counter PC, CPU registers).
L — Local Variables	Automatic local variables declared within procedure body.
T — Temporaries	Temporary intermediate expression evaluation registers/values.

🧠 M4 Active Recall & Exam-Important Question Bank

Q1. Explain the difference between Control Link (Dynamic Link) and Access Link (Static Link) in Activation Records. (8 Marks)

- **Control Link (Dynamic Link):** Points to the activation record of the *calling procedure* (who called me). It is determined strictly at runtime based on the dynamic execution call stack and is used to restore the caller's stack frame upon return.
- **Access Link (Static Link):** Points to the activation record of the *statically enclosing procedure* in the source code text (where was I lexically defined). It is determined at compile time based on lexical block nesting and is used by nested functions to access non-local variables.